

Rapport du projet

Ce projet consiste l'implémentation d'un jeu de société basé sur des tuiles avec des couleurs (Rouge , Bleu , Jaune , Noir) et des numéros entre 1 et 13 . Les joueurs piochent des tuiles depuis la pioche et forment des combinaisons (suites ou groupes), Le but est de vider sa main le plus vite possible .

Réalisé par :

- KHALDI Idir 22311085
- MAHIOU Riad 22413909

Types de données utilisés :

- **'a melt** : Représente un multi element (element , occurrence) .
- **'a mset** : Une liste de multi-éléments . [('a',2) ; ('b',3)] .
- **couleur** : les couleurs possibles des tuiles (Rouge, Bleu, Jaune, Noir).
- **tuile** : soit un Joker, soit une tuile colorée et numérotée.
- **main** : Un multi-ensemble de tuiles représentant la main d'un joueur.
- **pioche** : Un multi-ensemble de tuiles représentant la pioche.
- **table** : Un multi-ensemble de tuiles représentant les tuiles posées sur la table.
- **joueur** : Un enregistrement contenant le nom du joueur, sa main et son score.
- **combinaison** : Représente une combinaison valide de tuiles, soit une suite, soit un groupe.

les fonctions implémentées :

Des fonctions de base pour le multi-ensemble :

- **isEmpty** : Vérifie si un multi-ensemble est vide.

- **cardinal** : Calcule le nombre total de tuiles dans un multi-ensemble.
- **nbOcc** : Compte le nombre d'occurrences d'une tuile spécifique.
- **membre** : Vérifie si une tuile est présente dans un multi-ensemble.
- **subset** : Vérifie si un multi-ensemble est un sous-ensemble d'un autre.
- **add** : Ajoute une tuile à un multi-ensemble.
- **remove** : Supprime une tuile d'un multi-ensemble.
- **equal** : Compare deux multi-ensembles pour vérifier leur égalité.
- **sum** : Fusionne deux multi-ensembles en additionnant les multiplicités des tuiles communes.
- **intersection** : Calcule l'intersection de deux multi-ensembles.
- **difference** : Calcule la différence entre deux multi-ensembles.

Des fonctions spécifiques au jeu :

- **getRandom** : Sélectionne une tuile aléatoire dans un multi-ensemble.
- **combinaisons** : Détecte les combinaisons valides (suites ou groupes) dans une main.
- **creer_tuile** : Génère toutes les tuiles d'une couleur donnée.
- **remplire_pioch** : Initialise la pioche avec toutes les tuiles du jeu.
- **distribuer_cartes** : Distribue 14 tuiles à un joueur à partir de la pioche.
- **sum_tuiles** : Calcule le score d'une main en fonction des tuiles.
- **a_combinaison** : Vérifie si une main contient au moins une combinaison valide.

Des fonctions pour la gestion du jeu :

- **jouer** : Initialise le jeu, demande les noms des joueurs, et lance la boucle principale.

- **piocher** : Permet à un joueur de piocher une tuile de la pioche.
- **poser tuiles** : Permet à un joueur de poser des combinaisons de tuiles sur la table.
- **tour** : Gère le tour d'un joueur (pioche ou pose des tuiles).
- **Game loop** : Boucle principale du jeu, alternant les tours des joueurs jusqu'à la fin de la partie.

Pourquoi ces types et ces fonctions ?

la réponse est simple :

1. **Utilisation de multi-ensembles** : Permet de gérer facilement les tuiles et leurs multiplicités.
2. **Combinaisons détectées dynamiquement** : La fonction combinaisons parcourt la main pour détecter les suites et groupes, ce qui permet une flexibilité dans les règles du jeu.
3. **Gestion aléatoire de la pioche** : La fonction getRandom garantit que les tuiles sont piochées de manière aléatoire ..
4. **Structure modulaire** : Les fonctions sont découpées en petites unités logiques, ce qui facilite l'implémentation du code.

Répartition des tâches :

→ KHALDI Idir :

- ☒ Implémentation des fonctions de base pour les multi-ensembles .
(isEmpty, cardinal, nbOcc, etc.)
- ☒ Implémentation des fonctions de gestion du jeu .
(jouer, tour, game_loop).

→ MAHIOUT Riad :

- ☒ A testé les fonctions de base et la boucle principale du jeu.
- ☒ Implémentation des fonctions spécifiques au jeu .
(combinaisons, creer_tuile, remplir_pioch, distribuer_cartes, etc.)

Difficultés rencontrées et solutions adoptées

1. Détection des combinaisons :

Problème : La détection des suites et groupes était complexe à implémenter.

Solution : Utilisation de fonctions récursives et de filtres pour parcourir la main et identifier les combinaisons valides (on avait besoin d'un petit coup de main de notre chat "chat-gpt").

2. compilation du code sur VScode :

Problème : on a rencontré des problèmes pour exécuter notre codes sur VScode à cause des extensions OCaml .

Solution : Utilisation du compilateur en ligne TryOCaml .